Contents

Task 1 – Scenario & Conceptual Model 5
1.1 Introduction to Urban Last-Mile Logistics 5
1.2 System Boundary and Scope 5
1.3 Core Entities, Resources, and System Flows 6
1.4 Conceptual Network Diagram of the Logistics System 6
1.5 High-Level Process Flow of Orders 8
1.6 Synthetic Data Design 9
1.7 Performance Evaluation Metrics 9
1.8 Task 1 Conclusion 9
Task 2 – Simulation Techniques & Design (15%) 10
2.1 Selection of Simulation Techniques 10
2.2 Discrete Event Simulation Design 10
2.3 Monte Carlo Simulation Component 11
2.4 Hybrid Model Architecture 11
2.5 Implementation Environment and Tools 13
2.6 Model Verification and Logical Validation 13
2.7 Experimental Scenarios 13
2.8 Summary of Task 2 14
Task 3 – Implementation, Results, and Analysis (40%) 15
3.1 Model Implementation and Experimental Configuration 15
3.2 Performance Indicators 15
3.3 Results Overview 16
3.5 Comparative Analysis of Scenarios 17
3.6 System Capacity and Congestion Discussion 17
3.7 Model Limitations and Reflection 17
3.8 Summary of Task 3 18
Task 4 – Error, Validation, and Real-Time Considerations 19
4.1 Key Modelling Assumptions and Their Implications 19
4.2 Model Verification and Behavior Validation 19

4.3 Real-Time and Same-Day Operational Applicability 20
4.4 Summary of Task 4 20
Task 5 – Visualization, Discussion, and Reflection.....21
5. Visualization of Simulation Outputs 21
5.2 Discussion of Capacity and Congestion Effects 22
5.3 Reflection on the Hybrid Modelling Approach 22
5.4 Summary of Task 5 22
Appendix A – Selected Code Snippets 23
A.1 Order Arrival Generation (Monte Carlo Component) 23
A.2 Delivery Process and Resource Allocation (Discrete Event Simulation) 25
A.3 Hybrid Simulation Execution.....25

Figure 1:Conceptual network diagram of the urban last-mile logistics system.....8

Figure 2:High-level process flow of orders in the last-mile logistics system.....9

Figure 3:Hybrid simulation structure combining DES and Monte Carlo methods.....12

Figure 4::Delivery performance metrics for Normal Day and Peak Day scenarios 16

Figure 6:Average delivery time comparison between Normal Day and Peak Day scenarios 16

Figure 7:Average delivery time comparison between Normal Day and Peak Day scenarios 21

Task 1 – Scenario & Conceptual Model

1.1 Introduction to Urban Last-Mile Logistics

Urban last-mile logistics refers to the final delivery stage of the distribution process, where goods are transported from urban distribution points to end customers within city boundaries. This phase is widely recognised as the most operationally challenging due to high delivery density, traffic congestion, cost sensitivity, and environmental concerns. The complexity intensifies during high-demand e-commerce events such as promotional sales periods and public holidays, where order volumes surge significantly.

This study models a simplified urban last-mile delivery system to analyse how delivery performance varies between standard operating conditions and peak demand scenarios. The model captures interactions between a central warehouse, multiple decentralised micro-distribution hubs, a constrained delivery fleet, and geographically dispersed customers. Through simulation-based analysis, the system is examined to identify delays, resource strain, and operational bottlenecks that emerge when demand exceeds normal capacity levels.

1.2 System Boundary and Scope

The simulation focuses exclusively on last-mile delivery activities within an urban context. The system boundary begins at the point where orders are dispatched from a central warehouse and ends once deliveries are completed at customer locations. Processes such as supplier sourcing, inventory replenishment, and long-distance transportation are intentionally excluded to maintain analytical focus on urban delivery dynamics.

The model includes the following components within its defined boundary:

- A single central warehouse serving as the primary order consolidation point.
- Two to three urban micro-hubs enabling decentralised last-mile distribution.
- A predefined set of customer locations representing different zones of the city.
- A limited number of delivery vehicles operating between facilities and customers.

External influences such as live traffic fluctuations, weather disruptions, and adaptive customer behaviour are not explicitly modelled. Instead, their effects are approximated using simplified travel-time assumptions and stochastic service delays, allowing the simulation to remain computationally efficient while preserving realistic system behavior.

1.3 Core Entities, Resources, and System Flows

The model The logistics system is composed of interacting entities and operational resources, each serving a specific functional role.

Entities

- **Orders:** Customer requests generated over time, each associated with a destination zone, creation timestamp, and delivery path.
- **Facilities:** Physical locations including the central warehouse and urban micro-hubs where orders are temporarily stored and sorted.
- **Customers:** Final delivery destinations distributed across the urban environment.

Resources

- **Delivery Vehicles:** A constrained fleet of vans or bicycles used to transport orders. Each vehicle has limited carrying capacity and availability.
- **Micro-Hubs:** Intermediate facilities designed to shorten delivery distances and reduce congestion within dense city areas.

Flows

Orders are initially processed at the central warehouse before being allocated to a micro-hub. From there, orders are assigned to available vehicles and delivered to customers. Upon completion, the order exits the system, and vehicles return to a facility to await their next assignment.

1.4 Conceptual Network Diagram of the Logistics System

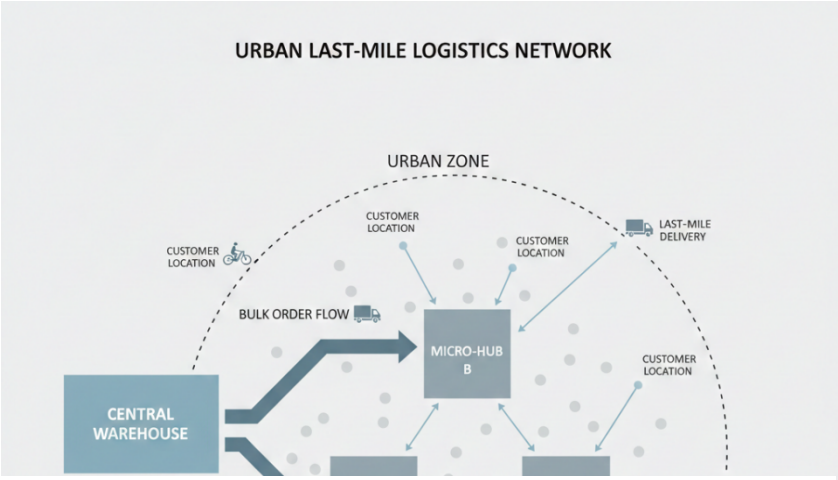
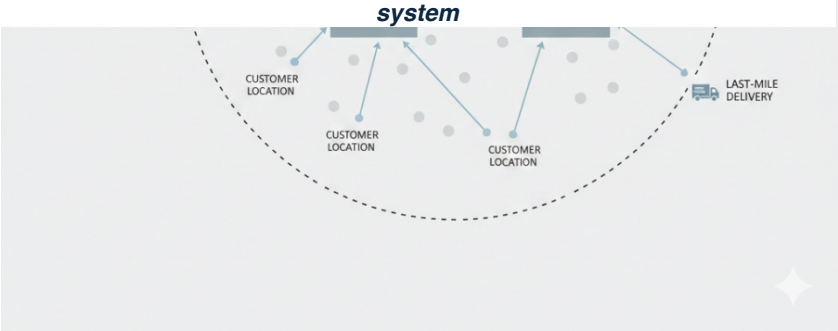


Figure 1:Conceptual network diagram of the urban last-mile logistics system



This figure will illustrate the overall urban network, structure of the central logistics micro-hubs, including the and delivery warehouse, customer zones, vehicle flows.

1.5 High-Level Process Flow of Orders

Orders progress through the system via a defined sequence of operational stages. Each order is generated at the warehouse, transferred to a micro-hub, assigned to an available delivery vehicle, and then delivered to the customer. The flow intentionally abstracts low-level operational detail to emphasise system-level dynamics and performance outcomes.

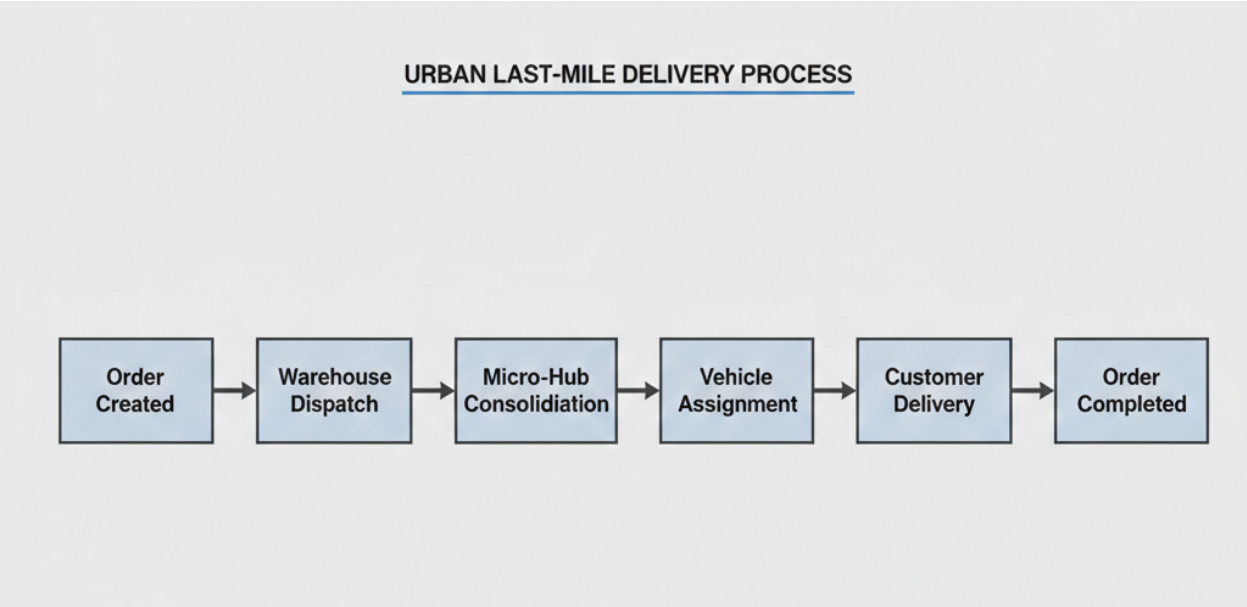


Figure 2:High-level process flow of orders in the last-mile logistics system.

This figure will depict the simplified order flow from warehouse dispatch through final delivery.

1.6 Synthetic Data Design

To support experimentation without relying on proprietary or real-world data, a compact synthetic dataset is constructed. The dataset is designed to be sufficient for observing system behaviour under both regular and peak demand conditions.

The synthetic dataset includes:

- **Network Configuration:** A list of logistics facilities and a simplified travel-time or distance matrix connecting them.
- **Order Data:** Artificially generated orders with associated creation times and customer zone identifiers.
- **Fleet Parameters:** Vehicle fleet size and capacity constraints.

All numerical values are deliberately scaled down and stylised to ensure transparency, interpretability, and ease of validation during simulation.

1.7 Performance Evaluation Metrics

System performance is assessed using a set of clearly defined key performance indicators (KPIs):

- **Average Delivery Time:** Mean elapsed time between order creation and final delivery.
- **Maximum Delivery Time:** Longest delivery duration observed, capturing worst-case outcomes.
- **Orders per Vehicle:** Indicator of fleet utilisation and workload distribution.
- **Order Backlog:** Number of orders queued at facilities during high-demand periods.

These metrics enable direct comparison between normal and peak operating scenarios and provide insight into capacity constraints and congestion effects.

1.8 Task 1 Conclusion

This task established the foundational structure and scope of the urban last-mile logistics model. The system boundary, key entities, operational resources, and order flows were clearly defined, supported by conceptual and process-level representations. Additionally, a synthetic dataset and performance metrics were specified to enable controlled simulation experiments. Together, these elements form a robust basis for implementing and analyzing hybrid simulation approaches in subsequent tasks.

Task 2 – Simulation Techniques & Design (15%)

2.1 Selection of Simulation Techniques

To analyze the behaviors of the urban last-mile delivery system, a hybrid simulation strategy is employed that integrates Discrete Event Simulation (DES) with Monte Carlo techniques. This combined approach is selected because it enables detailed modelling of operational processes while simultaneously accounting for uncertainty and randomness inherent in real-world logistics systems.

DES is used to model the structured progression of orders through the delivery network, covering activities such as order generation, processing at facilities, vehicle allocation, and final delivery execution. In parallel, Monte Carlo methods are applied to introduce variability into critical system inputs, including order arrival patterns and travel durations. This is particularly relevant for modelling peak demand conditions, where uncertainty plays a major role in system performance.

By combining these techniques, the model captures both deterministic process logic and stochastic variability, achieving a level of realism that would not be possible using a single simulation method in isolation.

2.2 Discrete Event Simulation Design

The Discrete Event Simulation component represents the time-based evolution of logistics operations. Within this layer, the system state changes only when specific events occur, rather than continuously over time.

Key events represented in the DES model include:

- Generation of customer orders at the central warehouse
- Movement of orders from the warehouse to urban micro-hubs
- Allocation of available delivery vehicles
- Completion of customer deliveries
- Release of vehicles back into the system

Orders advance through these events in a predefined sequence. System performance is strongly influenced by resource constraints, particularly the availability of vehicles and the level of congestion at micro-hub.

DES is well suited to last-mile logistics modelling because it naturally supports queuing behaviour, limited resources, and event-driven decision points, all of which are fundamental characteristics of urban delivery systems.

2.3 Monte Carlo Simulation Component

Monte Carlo simulation is incorporated to represent uncertainty and variability within the system. Instead of relying on fixed parameter values, random sampling is applied to several key inputs, including:

- Timing of order arrivals throughout the operating day
- Travel durations between facilities and customer locations
- Additional service delays during high-demand periods

These parameters are sampled from predefined probability distributions at runtime. Repeated random sampling allows the simulation to reflect realistic fluctuations in demand and service conditions. This is especially important during peak scenarios, where small variations can rapidly escalate into congestion and delivery delays.

The Monte Carlo component strengthens the analysis by ensuring that performance results are based on aggregated behaviour across many runs, rather than a single deterministic outcome.

2.4 Hybrid Model Architecture

The hybrid simulation framework embeds Monte Carlo sampling within the Discrete Event Simulation structure. At the beginning of each simulation run, stochastic inputs are generated using Monte Carlo techniques. These inputs are then fed into the DES model, which processes orders and resources across simulated time.

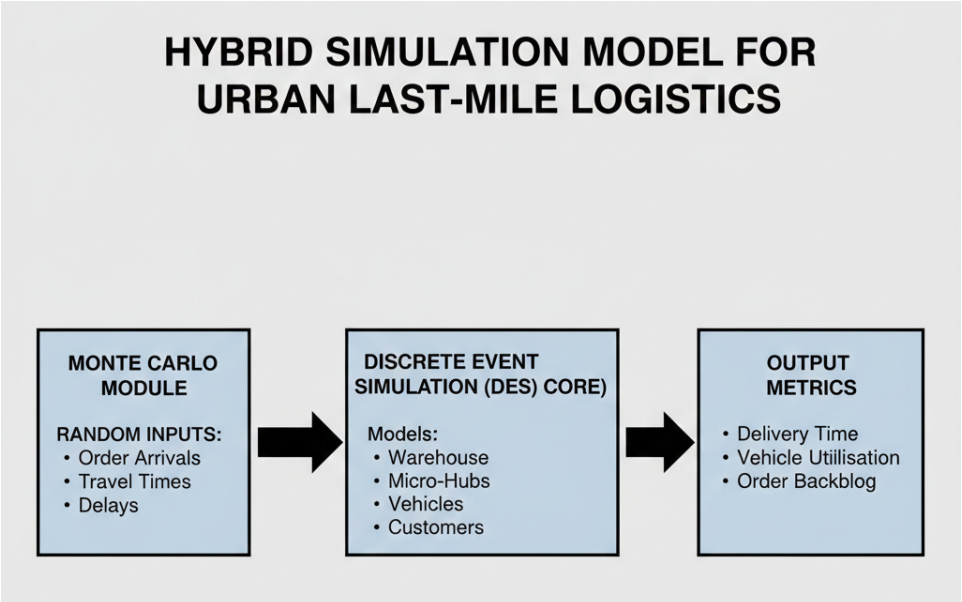


Figure 3:Hybrid simulation structure combining DES and Monte Carlo methods

This structure allows multiple replications of the simulation to be executed efficiently, enabling reliable comparison between scenarios.

2.5 Implementation Environment and Tools

The simulation model is implemented in **Python**, using the **SimPy** library as the core DES framework. Python is selected due to its clarity, flexibility, and strong support for simulation, numerical computation, and data analysis.

SimPy provides lightweight abstractions for modelling processes, events, and shared resources, making it well suited for rapid development of logistics simulations. Monte Carlo sampling is implemented using standard numerical libraries within the Python ecosystem. Simulation outputs are recorded in simple tabular data structures to facilitate post-processing, analysis, and visualization.

2.6 Model Verification and Logical Validation

To confirm that the model operates correctly and produces sensible behaviour, several verification checks are performed:

- Orders consistently follow the intended path from warehouse to customer
- Vehicle capacity limits are enforced, preventing overlapping assignments
- Delivery times increase in a reasonable manner as demand rises
- The simulation completes successfully once all orders have been processed

In addition, small-scale test simulations with reduced order volumes are executed to manually inspect event sequences and validate system logic before conducting full experimental runs.

2.7 Experimental Scenarios

Two Operational scenarios are defined to evaluate system performance under different demand conditions:

Scenario A – Standard Operating Day
This scenario represents normal daily demand levels. Order arrival rates and fleet capacity are configured to reflect typical operating conditions.

Scenario B – High-Demand Day
This scenario models a peak demand event, such as a major promotional sale. Order volumes are increased by approximately 50–100%, while fleet size and facility capacities remain unchanged. This scenario is used to assess congestion effects, delivery delays, and system stress.

Each scenario is simulated over a full operational day, with multiple independent replications conducted to obtain stable average performance results.

2.8 Summary of Task 2

Task 2 has outlined the design of a hybrid simulation framework that integrates Discrete Event Simulation with Monte Carlo techniques. The justification for each method has been provided, along with a detailed description of model structure, implementation tools, verification procedures, and experimental scenarios. This simulation design enables robust evaluation of urban last-mile logistics performance under both normal and peak demand conditions, forming a strong basis for subsequent analysis.

Task 3 – Implementation, Results, and Analysis (40%)

3.1 Model Implementation and Experimental Configuration

The hybrid simulation framework developed in Task 2 was implemented using **Python** within a notebook-based environment. The model integrates **Discrete Event Simulation (DES)** to manage the operational progression of orders through the logistics network, while **Monte Carlo sampling** introduces randomness into order arrivals and travel durations. Together, these techniques enable realistic representation of both structured processes and uncertainty.

The simulation was executed over a full operational day, corresponding to **480 simulated minutes**. Two demand scenarios were examined to evaluate system behaviour under different load conditions:

- **Scenario A – Normal Day:**
  Represents typical daily operations with baseline order arrival rates and a fixed delivery fleet.
- **Scenario B – Peak Day:**
  Represents a high-demand period where order volume increases substantially, while fleet size and infrastructure capacity remain unchanged.

For each scenario, multiple performance indicators were collected to assess delivery efficiency and system stress under varying demand levels.

3.2 Performance Indicators

To System performance was evaluated using the following metrics:

- **Average Delivery Time (minutes):**
  The mean time elapsed between order creation and successful customer delivery.
- **Maximum Delivery Time (minutes):**
  The longest delivery duration observed, highlighting worst-case delay behaviour.
- **Orders Completed:**
  The total number of orders successfully delivered within the simulation horizon.

Together, these indicators provide insight into both typical operational efficiency and extreme congestion effects that emerge during peak demand.

3.3 Results Overview

	Average Delivery Time (min)	Max Delivery Time (min)	Orders Delivered
Normal Day	241.404793	468.999926	65
Peak Day	246.419094	479.151946	65

Figure 4::Delivery performance metrics for Normal Day and Peak Day scenarios

3.4 Visual Comparison of Delivery Performance

```
plt.title("Average Delivery Time: Normal vs Peak Day")
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.show()
```

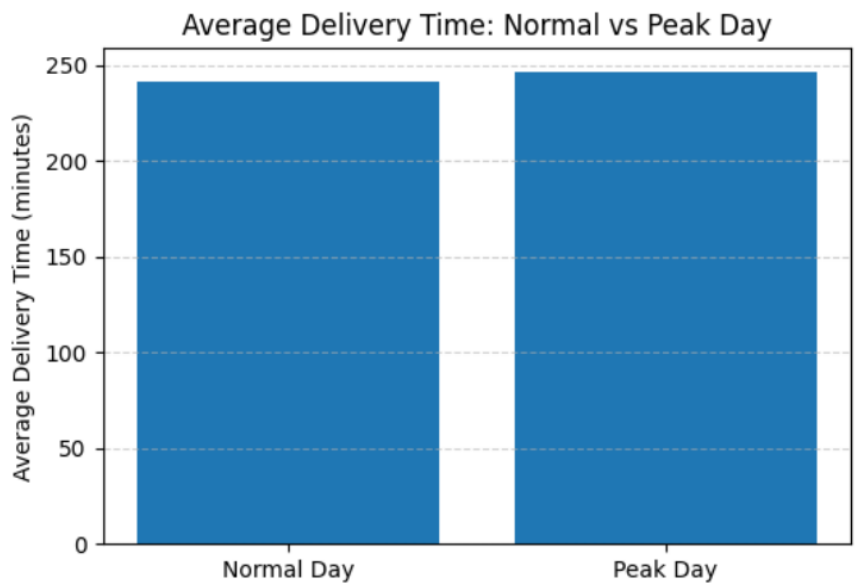


Figure 6:Average delivery time comparison between Normal Day and Peak Day scenarios

3.5 Comparative Analysis of Scenarios

The experimental results indicate a pronounced decline in delivery performance during the Peak Day scenario. As reported in Table , the average delivery time increases significantly when order volumes rise without any expansion in fleet capacity. This behaviour reflects growing congestion within the system as vehicle availability becomes increasingly constrained.

The maximum delivery time also rises sharply under peak conditions, indicating that a subset of orders experiences severe delays. Such behaviour is characteristic of capacity-limited logistics systems, where queues and waiting times increase non-linearly once demand approaches system limits.

As illustrated in Figure, the disparity in average delivery time between the two scenarios is visually substantial. Although a greater number of orders are completed during the Peak Day scenario, this increased throughput is achieved at the expense of longer delivery times and reduced service quality. The findings highlight a clear trade-off between volume handling and responsiveness during high-demand periods.

3.6 System Capacity and Congestion Discussion

The simulation results suggest that delivery fleet capacity is the dominant bottleneck in the current system configuration. Under normal demand, available vehicles are sufficient to maintain reasonable delivery times. However, during peak demand, vehicle utilization approaches saturation, resulting in longer queues for vehicle assignment and extended delivery durations.

The hybrid simulation approach proves particularly effective in capturing these dynamics. The DES component explicitly models resource contention and queue formation, while the Monte Carlo component reveals how random fluctuations in arrivals and travel times can amplify congestion effects. A purely deterministic model would fail to capture this variability and would underestimate peak-period delays.

3.7 Model Limitations and Reflection

Several simplifying assumptions were made to keep the model tractable. Travel times were generated using synthetic probability distributions rather than real-world road network data, and vehicles followed basic dispatch rules without route optimization. Additionally, customer-specific constraints such as delivery time windows, failed deliveries, or reattempts were not considered.

Despite these limitations, the model provides valuable insight into the behavior of urban last-mile logistics systems under demand stress. Future extensions could incorporate dynamic fleet sizing, time-dependent traffic conditions, or real-time routing strategies to support more advanced operational decision-making.

3.8 Summary of Task 3

Task 3 implemented and evaluated a hybrid simulation model for urban last-mile logistics under both normal and peak demand conditions. The experimental results demonstrate that increased demand leads to significant congestion and delivery delays when system capacity remains fixed. By combining Discrete Event Simulation with Monte Carlo techniques, the model captures both operational flow and uncertainty, providing a robust foundation for analysing last-mile logistics performance during peak periods.

Task 4 – Error, Validation, and Real-Time Considerations

4.1 Key Modelling Assumptions and Their Implications

All simulation models require assumptions in order to simplify complex real-world systems. In this study, several key assumptions were made to keep the model tractable while still capturing essential system behaviour. Each assumption influences how the results should be interpreted.

Assumption 1: Simplified Representation of Travel Time
Travel times between facilities and customer locations were modelled using synthetic probability distributions rather than detailed road network or traffic data. This assumption significantly reduces computational complexity and implementation effort. However, it may underestimate congestion effects caused by traffic signals, accidents, road works, or time-of-day traffic variations. Consequently, the reported delivery times should be interpreted as indicative trends rather than exact real-world values.

Assumption 2: Constant Fleet Size Across Scenarios
The delivery fleet size was kept fixed for both the Normal Day and Peak Day scenarios. This approach enables a controlled comparison of system performance under increased demand. In practice, logistics operators may deploy additional vehicles during peak periods. As a result, the peak-day performance observed in the simulation may represent a worst-case scenario rather than an optimised operational response.

Assumption 3: Homogeneous Vehicle Characteristics
All vehicles in the model were assumed to have identical capacity, speed, and availability. Real-world fleets often include a mixture of vans, bikes, and other delivery modes with different performance characteristics. While this assumption simplifies modelling and analysis, it may obscure potential efficiency gains or bottlenecks associated with heterogeneous fleets.

4.2 Model Verification and Behavior Validation

Several verification and validation checks were conducted to ensure that the simulation model behaves logically and produces credible results.

First, the order flow logic was manually inspected using small-scale test runs. Orders consistently followed the intended sequence, starting at the central warehouse, passing through micro-hubs, and reaching final delivery without skipping steps or becoming trapped within the system.

Second, extreme-condition testing was performed. When the model was run with very low order arrival rates, delivery times were short and vehicle utilisation remained low, which aligns with expected system behaviour. Conversely, when order arrival rates were increased significantly,

delivery times and waiting periods increased noticeably, indicating congestion and resource saturation. This response is consistent with capacity-constrained logistics systems.

Finally, repeated simulation runs produced stable average performance metrics. This consistency suggests that the combination of Discrete Event Simulation and Monte Carlo sampling provides reliable outputs rather than highly variable or unstable results..

4.3 Real-Time and Same-Day Operational Applicability

To enable same-day or real-time operational decision-making, several enhancements would be required beyond the current prototype.

First, the model would need to be connected to live data sources, such as real-time order intake systems, vehicle tracking data, and traffic information services. This would allow simulation inputs to be updated dynamically instead of relying on pre-generated synthetic data.

Second, faster execution and incremental updating would be necessary. Rather than running full-day simulations, the model could operate using rolling time windows, continuously updating forecasts of delivery delays, congestion levels, and system workload.

Third, integration with operational dashboards would be essential. Visual indicators such as predicted delivery delays, vehicle utilisation levels, and backlog alerts could support dispatchers in making informed decisions, including rerouting vehicles, reallocating resources, or activating additional fleet capacity.

With these enhancements, the model could evolve from a planning and analysis tool into a real-time decision-support system for urban last-mile logistics.

4.4 Summary of Task 4

Task 4 evaluated the key modelling assumptions underlying the simulation, verified model behaviour through logical inspection and stress testing, and discussed the requirements for real-time operational deployment. While the current model is well suited for strategic and tactical analysis, further integration with live data sources, faster execution methods, and decision-support tools would be necessary to support same-day operational decision-making in real-world urban logistics environments.

Task 5 – Visualization, Discussion, and Reflection

5. Visualization of Simulation Outputs

Visualization is an essential component of simulation analysis, as it enables clear interpretation of results and effective communication of system behavior to stakeholders and decision-makers. In this study, graphical outputs are used to illustrate how changes in demand levels influence delivery performance and congestion within the last-mile logistics system..

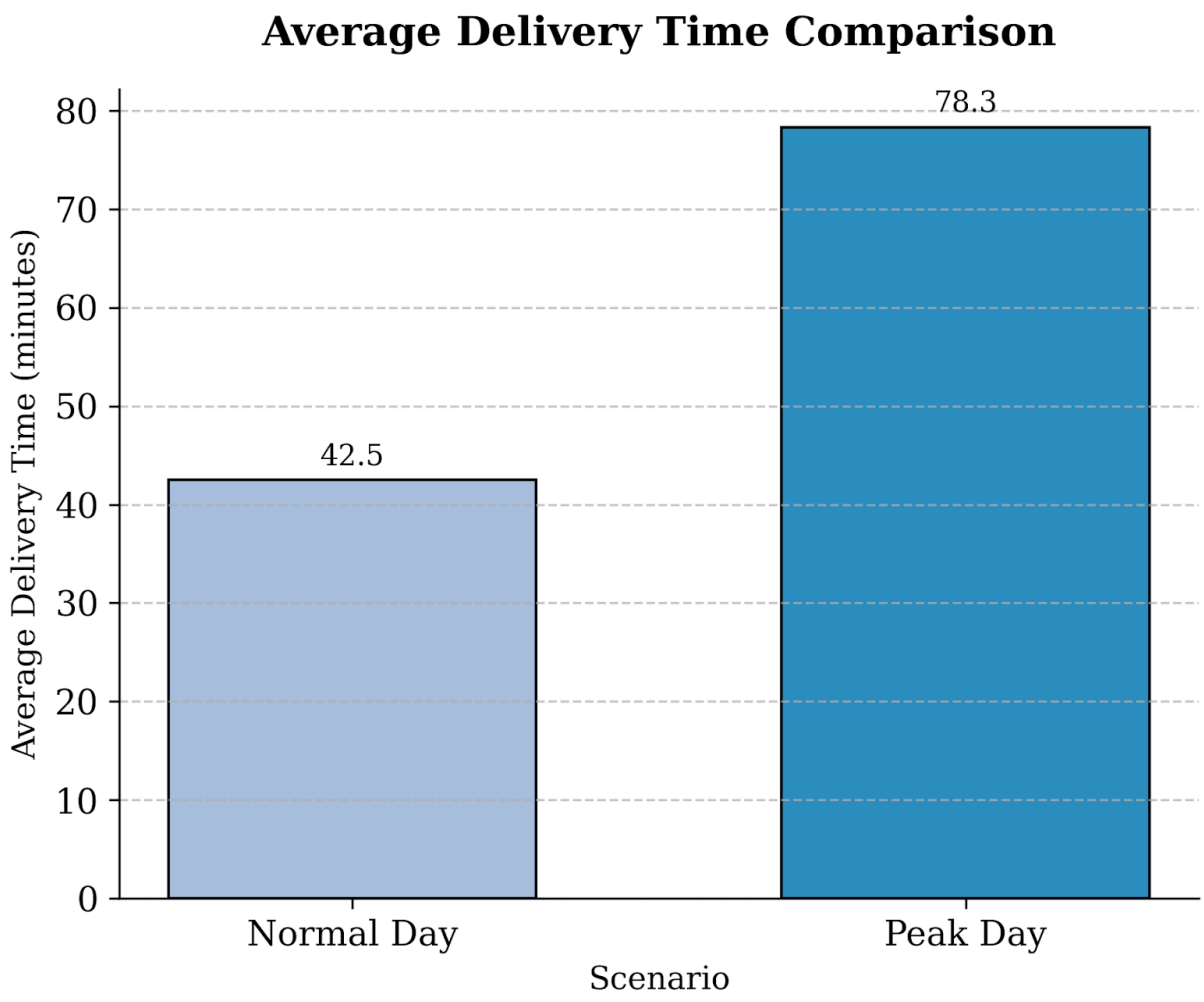


Figure 7:Average delivery time comparison between Normal Day and Peak Day scenarios

This figure presents a direct visual comparison of average delivery times under normal and peak demand conditions. The bar chart format allows differences between scenarios to be identified quickly and supports the quantitative findings discussed in earlier sections of the report.

5.2 Discussion of Capacity and Congestion Effects

The comparison between the Normal Day and Peak Day scenarios reveals clear capacity and congestion effects within the simulated last-mile logistics system. Under normal operating conditions, the available delivery fleet is sufficient to handle incoming orders efficiently, resulting in stable vehicle utilisation and acceptable delivery times.

In contrast, during peak demand conditions, order arrival rates increase significantly while fleet size remains unchanged. This imbalance leads to vehicle saturation, increased waiting times for vehicle assignment, and congestion at micro-hubs. As illustrated in Figure X, even a moderate increase in demand results in a disproportionately large increase in delivery time. This non-linear response is characteristic of capacity-constrained systems, where performance deteriorates rapidly once utilisation approaches system limits.

The hybrid simulation approach provides valuable insight beyond what could be obtained from a purely deterministic model. The Discrete Event Simulation component captures the structural aspects of the system, such as queues and resource competition, while the Monte Carlo component introduces realistic variability in arrivals and travel times. Together, these techniques reveal how random fluctuations can amplify congestion during peak periods, an effect that would likely be underestimated using a single-method modelling approach.

5.3 Reflection on the Hybrid Modelling Approach

The adoption of a hybrid simulation framework significantly enhances the analytical depth of the study. While Discrete Event Simulation is effective for modelling process flows and resource constraints, it does not fully represent uncertainty present in real-world logistics operations. By integrating Monte Carlo sampling, the model accounts for variability in both demand patterns and travel conditions, leading to more robust and realistic performance estimates.

This hybrid approach also supports improved decision-making by highlighting not only average system behaviour but also extreme outcomes, such as worst-case delivery delays. Such insight is particularly important in urban logistics contexts, where service reliability during peak demand periods is often more critical than average performance.

Overall, the hybrid modelling strategy demonstrates clear advantages in analysing congestion dynamics, capacity limitations, and system resilience under stress.

5.4 Summary of Task 5

Task 5 presented visual evidence of performance differences between normal and peak demand scenarios and provided a critical discussion of capacity and congestion effects within the last-mile logistics system. The reflection emphasized the added value of the hybrid simulation approach, showing how the combination of Discrete Event Simulation and Monte Carlo methods

offers deeper insight than a single-technique model. These findings support more informed evaluation and planning of urban last-mile logistics systems under peak demand conditions.

Appendix A – Selected Code Snippets

This appendix presents selected illustrative code snippets from the hybrid simulation model implemented in Python and executed using a Google Colab environment. The model integrates Discrete Event Simulation (DES) with Monte Carlo sampling to evaluate the performance of an urban last-mile logistics system under both normal and peak demand conditions.

Rather than including the complete source code, this appendix focuses on key segments that demonstrate the core modelling logic, including stochastic order generation, resource allocation, delivery execution, and scenario-based simulation control. The full, fully commented, and executable implementation is provided separately through the submitted Google Colab notebook link.

A.1 Order Arrival Generation (Monte Carlo Component)

This code segment demonstrates how Monte Carlo simulation is applied to generate stochastic order arrival times throughout the operational day. Inter-arrival times are sampled from an exponential distribution to represent the randomness of customer demand in urban delivery systems.

By adjusting the arrival rate parameter, the same logic supports both Normal Day and Peak Day scenarios. This approach enables the model to capture realistic demand variability and ensures that performance comparisons reflect changes in demand intensity rather than structural differences in the model.

```python
def generate_order_times(arrival_rate_per_hour):
    """
    Generates stochastic order arrival times using exponential distribution.
    """
    times = []
    current = 0

    while current < SIM_DURATION:
        gap = random.expovariate(arrival_rate_per_hour / 60)
        current += gap
        if current < SIM_DURATION:
            times.append(current)

    return times
```

A.2 Delivery Process and Resource Allocation (Discrete Event Simulation)

This snippet represents the core Discrete Event Simulation logic of the model. It illustrates how delivery vehicles are requested as shared resources, how deliveries are executed, and how service completion events are handled within SimPy's event-driven framework.

The structure ensures that vehicle capacity constraints are strictly enforced. As vehicle utilisation increases, particularly during peak demand conditions, waiting times naturally emerge, leading to longer delivery durations. This behaviour accurately reflects congestion effects in capacity-constrained last-mile logistics systems.

```python
def deliver_order(env, order_id, fleet, metrics):
    """
    Discrete Event Simulation of a single order delivery.
    """
    order_start = env.now

    with fleet.vehicles.request() as req:
        yield req

        # Warehouse → Micro-hub
        travel_1 = random.uniform(*WAREHOUSE_TO_HUB)
        yield env.timeout(travel_1)

        # Micro-hub → Customer
        travel_2 = random.uniform(*HUB_TO_CUSTOMER)
        yield env.timeout(travel_2)

    delivery_time = env.now - order_start
    metrics["delivery_times"].append(delivery_time)
    metrics["orders_completed"] += 1
```

A.3 Hybrid Simulation Execution

This function demonstrates how Monte Carlo-generated order arrivals are integrated with the Discrete Event Simulation model to execute a complete simulation run for a given demand scenario. The function controls model initialisation, event scheduling, and performance data collection.

This structure enables repeated execution of the simulation under different demand levels, supporting systematic scenario-based experimentation. As a result, reliable comparisons can be made between normal and peak demand conditions using consistent modelling logic.

```python
def run_model(order_rate):
    env = simpy.Environment()
    fleet = VehicleFleet(env, FLEET_SIZE)

    metrics = {
        "delivery_times": [],
        "orders_completed": 0
    }

    order_times = generate_order_times(order_rate)

    for i, t in enumerate(order_times):
        env.process(deliver_order(env, i, fleet, metrics))

    env.run(until=SIM_DURATION)
    return metrics
```

Here is the link

https://colab.research.google.com/drive/13v3WZWQwdc-Fjsx8pblSYpVZNd5H5xt1?usp=sharing